

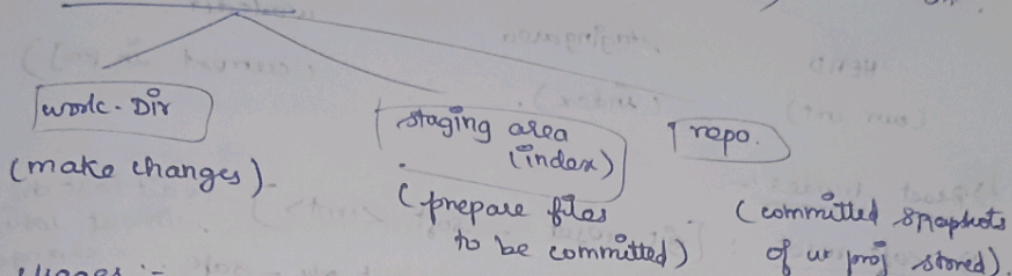
10/6/26

Undoing changes:-

1) git checkout :- $\rightarrow$ switch branches / restore files.

$\rightarrow$ revert changes in working dir to state of spec. commit.
(only affects work dir not other 2)

Git obj model:



Usages :-

① `$ git checkout <file>` - replace file to last com. version from cur branch.

- Irreversible $\rightarrow$ uncommitted changes lost.
 $\rightarrow$ so, git refuses, $\Rightarrow$ use `-f` - force flag here.

② `$ git checkout <commit-hash> -- <file>` - specific hash.
 $\downarrow$ find from git log. $\downarrow$ distinguishes b/w hash & filename.

usecase :-

- discard local changes
- restore prev version
- revert merged changes.

2) git restore :- $\rightarrow$ git 2.23 version introduced.

$\rightarrow$ more focused & user friendly than checkout.
 $\rightarrow$ works on all 3 obj models.

Usage :-

① In work dir : In spec. files and discard the changes. [to last commit]
In 1 file $\rightarrow$ `$ git restore <file>`

In mult files $\rightarrow$ `$ git restore <file1> <file2> <file3>`

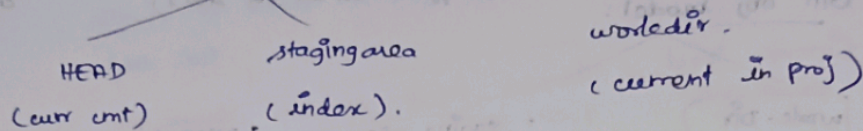
② staged changes : modify bef commit. `$ git restore --staged <file>`

③ to spec commit : `$ git restore --source <commit> <file>` } multiple files also given to unstaged.

④ combined with : `$ git restore --source <commit> --worktree <file>` } reflects state of work dir.

`$ git restore --source <commit> -- <dir> /*` } all files in spec. dir.

- 3) git reset: move curr branch pointer to diff cnt.
 ↳ alters cnt history of branch - directly HEAD ptr.
 ↳ modify



↳ Reset modes:-

- 1) soft reset: `[git reset --soft <cnt>]` - (edit last cnt without losing changes)
 ↳ moves HEAD to spec cnt only - safe.
- 2) mixed: `[git reset <cnt> / git reset --mixed <cnt>]` (refine committing)
 ↳ moves HEAD to spec cnt &
 → reset staging area to match it.
 (when want to stage & keep in work dir)
- 3) hard: `[git reset --hard <cnt>]` (when confident that they are no longer needed)
 ↳ moves HEAD to spec cnt.
 ↳ restages area.
 ↳ alters working dir.
 } destructive op.

↳ Recover:-

- ① Reflog: tracks HEAD where has been.

`$ git reflog` → identify hash & reset back with git reset.

- ② git revert: → undo the changes that spec. cnt made and make a new commit. - transparent.

↳ usage: ① `$git revert <cnt hash>`

↳ open def editor & aske fr msg. - def reverted <hash>

② mult. files: `$git revert HEAD~2..HEAD` - spec. range of commits.

③ merge commit: → specify which parent commit want to use as reference pt fr revert.
`$git revert -m <parent-no> <merge-cnt hash>`

④ if conflicts → resolve & `$git revert --continue`

→ if don't want → `$git revert --abort`

HEAD

- 5) git clean:- → remove untracked files from work. dir.
→ (eg): temp files created dur dev.
Build artifacts.
config files.
→ use after git status to be effective.

↳ common options:

- ① -f / --force → without this, git only shows what ^{would} be deleted.
- ① -d / --dir → removes untracked directories with files.
- ① -x / --ignore → rem files ignored by .gitignore. (cautious)
- ① -n / --dry-run → shows what would be deleted without actually delete. (safer one)

- 6) git stash:- → safety stores the unfinished changes sep. stack
→ when runs, create new stash.

↳ working:

- (i) create new stash entry :- as stash obj.
- in .git dir → refs/stash.

- (ii) after created, resets work dir to, ^{reverted after} ptr to latest stash.
last commit. - fr clean state.

↳ usages:

- ① \$git stash → saves modified & staged changes.
- ① \$git stash -u → both tracked & untracked changes.
- ① List → \$git stash list → ^{dep with index}
- ① apply → \$git stash apply → recent one.
\$git stash apply stash@{1} ^{indexed one, stashed.} } after applied, remove this not autoremoved. \$git stash pop.
- ① drop → \$git stash drop stash@{0}
- ① clear → \$git stash clear → all stashed cleared.
- ① before applying, check what bring back : \$git stash show -p stash@{0}
- ① give msg : \$git stash save "msg"